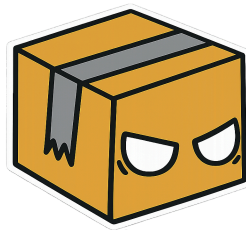


SmartLogix – Plataforma Inteligente para la Gestión Logística de eCommerce.

Diseño de Arquitectura y Patrones de Microservicios.



Sección: 003D.

Docente: Claudio Esteban Rojas Rodriguez.

Integrantes:

- **Gabriel Jeria.**
- **Vicente Palacios.**
- **Isak Chacana.**

Introducción.

Contexto.

El auge del comercio electrónico ha impulsado la necesidad de sistemas más eficientes y escalables para la gestión de inventarios, pedidos y envíos. SmartLogix es una startup en crecimiento que busca ofrecer una plataforma moderna e integrada para la optimización de procesos logísticos en pequeñas y medianas empresas (PYMEs) de eCommerce. La empresa ha identificado que muchas de estas PYMEs aún dependen de sistemas monolíticos y procesos manuales, lo que genera inconsistencias en la información, retrasos en la entrega y dificultades para manejar picos de demanda.

Para resolver estos problemas, SmartLogix quiere desarrollar una solución basada en microservicios, con una capa frontend moderna y flexible, y un backend escalable y seguro, permitiendo a las PYMEs gestionar su operación logística con mayor eficiencia y reducir costos operativos.

Actualmente, las empresas que utilizan soluciones logísticas tradicionales enfrentan desafíos como sincronización deficiente de inventarios, errores en la asignación de pedidos, y demoras en la coordinación de envíos. Muchos de estos problemas surgen porque sus sistemas monolíticos no permiten la automatización ni integración con nuevas tecnologías. SmartLogix ha decidido desarrollar una arquitectura moderna basada en microservicios para garantizar la escalabilidad y flexibilidad de su plataforma, facilitando la automatización de la gestión logística.

Desarrollo

Arquitectura

El sistema SmartLogix está compuesto por un componente Frontend desarrollado en React + Vite, un Backend For Frontend (BFF) encargado de centralizar las peticiones del cliente y de múltiples microservicios y 4 servicios hechos en Java con framework de Spring Boot. Cada microservicio posee su propia base de datos, continuando con el principio de desacoplamiento y autonomía de datos.

Todo este sistema está implementado con Kubernetes, el cual ajusta el escalado de cada servicio dependiendo de su carga

Además, la comunicación entre los componentes se realiza mediante APIs REST, permitiendo una integración flexible y facilitando futuras ampliaciones del sistema.

Se usa Flyway para el manejo de esquemas dentro de cada base de datos de cada microservicio, asegurando la consistencia y robustez de la base de datos y el apropiado control de versionado dentro de esta.

También los microservicios tienen una implementación de repository pattern, con Java Persistence API (JPA) el cual interactúa con su base de datos PostgreSQL correspondiente, el cual esta alojado en un Persistent Volume Claim (PVC) dentro de Kubernetes.

Las pruebas unitarias son realizadas a través de junit, los cuales verifican la integridad del código a la hora de desplegarlo.

API Gateway

El diseño considera el uso de un API Gateway para centralizar el acceso a los servicios del sistema. Este componente permite gestionar rutas, autenticación, seguridad y monitoreo de solicitudes realizadas hacia los distintos microservicios. Para este proyecto usamos KrakenD como nuestro gateway, asegurando que cualquier conexión no segura sea permitida, y aislando el backend del resto de internet.

Frontend

El Frontend está desarrollado con Typescript con el ambiente Node JS, usando las librerías de React y empaquetada con Vite, este sigue un modelo de arquitectura en capas, separando sus obligaciones en Api (capa de servicio), ViewModels (capa lógica) y pages (capa vista o visual). Esto permite separar la lógica de presentación de la interfaz gráfica, facilitando la reutilización y mantenimiento de componentes.

El Frontend tiene un pantalla de ingreso y autentifica la identidad del usuario, tras eso procede tener una pantalla de inventario, órdenes y pedidos, los cuales visualizan y crean pedidos de API, interactuando con el Backend a través de la API.

API Gateway (KrakenD)

Durante el desarrollo, el Backend for Frontend (BFF) implementado en Spring Boot fue reemplazado por KrakenD como API Gateway declarativo. A diferencia del BFF anterior, KrakenD no ejecuta código propio: se configura mediante un archivo krakend.json que define, por cada endpoint público, a qué microservicio debe enrutar la petición. KrakenD centraliza la validación de los tokens JWT (algoritmo HS256) mediante su módulo auth/validator, y aplica circuit breaker por endpoint para tolerar fallas de los microservicios backend. La generación del token JWT, que antes realizaba el BFF, se trasladó al microservicio ms-user, el cual lo emite al momento

del login usando la misma clave secreta que KrakenD utiliza para validarlo. Este cambio simplifica el mantenimiento de la capa de gateway, reduciendo código propio a cambio de configuración declarativa.

Users

El microservicio ms-user se encarga de almacenar y gestionar los usuarios dentro del sistema, además de en conjunto con el BFF, ayuda a la autenticación de estos mismos.

Este microservicio sigue el arquetipo Modelo-Servicio-Controlador (MSC): la capa Controller expone los endpoints REST bajo /api/users, la capa Service concentra la lógica de negocio y validaciones, y la capa Repository (Spring Data JPA) abstrae el acceso a la base de datos PostgreSQL propia del servicio (user-db), siguiendo el principio de autonomía de datos por microservicio.

Inventory

Este se encarga de manejar los productos dentro del inventario, manteniendo el stock, avisando cuando haya stock bajo y el precio de cada producto.

Este microservicio sigue el arquetipo Modelo-Servicio-Controlador (MSC): la capa Controller expone los endpoints REST bajo /api/inventory, la capa Service concentra la lógica de negocio y validaciones, y la capa Repository (Spring Data JPA) abstrae el acceso a la base de datos PostgreSQL propia del servicio (inventory-db), siguiendo el principio de autonomía de datos por microservicio.

Orders

Este se encarga del manejo de órdenes realizadas por los usuarios de un marketplace, incluyendo estado, dirección, a quien se dirige y el total de la orden. La validación de stock disponible y su descuento automático al crear una orden formaban parte de la lógica del BFF anterior. Tras la migración a KrakenD como API Gateway declarativo, esta validación quedó pendiente de reintegrar directamente en ms-orders, como parte de las siguientes iteraciones del proyecto.

Este microservicio sigue el arquetipo Modelo-Servicio-Controlador (MSC): la capa Controller expone los endpoints REST bajo /api/orders, la capa Service concentra la lógica de negocio y validaciones, y la capa Repository (Spring Data JPA) abstrae el acceso a la base de datos PostgreSQL propia del servicio (orders-db), siguiendo el principio de autonomía de datos por microservicio.

Restock

El microservicio ms-restock gestiona las solicitudes de reposición de stock de la plataforma SmartLogix. Permite crear, consultar, aprobar, rechazar y eliminar

solicitudes asociadas a un ítem y bodega , manteniendo un ciclo de vida controlado mediante el atributo estado ; PENDIENTE, APROBADA, RECHAZADA y COMPLETADA.

Sigue el arquetipo MSC, con persistencia en una base de datos PostgreSQL propia, gestionada mediante Flyway, respetando el principio de autonomía de datos por microservicio .La API REST se expone en el puerto 9093 bajo /api/restock.

Se integra con ms-inventory mediante RestTemplate: al crear una solicitud válida que el ítem existe en el inventario , y al probarla actualiza automáticamente el stock sumando la cantidad solicitada, dándole coherencia real al flujo de negocio.

El BFF consume este servicio a través de un cliente Feign (RestockServiceClient) y lo expone bajo /api/bff/restock hacia frontend y el API Gateway KrakenD. Las pruebas unitarias se implementan con JUnit y Mockito sobre la capa de servicio , cubriendo los casos principales del ciclo de vida de una solicitud con una cobertura superior al 60%.

Integración de servicio de correo transaccional: API de Brevo.

Para cumplir con el requisito de notificación automática a los usuarios, se integró el sistema con Brevo (anteriormente conocido como Sendinblue), un proveedor de servicios de correo electrónico transaccional que expone una API REST para el envío programático de correos. Se eligió esta solución por ofrecer un plan gratuito de 300 correos diarios sin necesidad de tarjeta de crédito, lo cual la hace adecuada para un proyecto académico sin presupuesto asociado.

La API de Brevo se utiliza específicamente en el flujo de registro de usuarios del microservicio ms-user. Su función es enviar un correo electrónico de bienvenida automáticamente cada vez que un nuevo usuario completa su registro en la plataforma, confirmándole que su cuenta fue creada exitosamente.

Pruebas unitarias en ms-restock y BFF

Para ms-restock se implementaron pruebas unitarias con JUnit 5 y Mockito sobre las capas de Service y Controller, cubriendo la creación, consulta, aprobación, rechazo y eliminación de solicitudes, así como las validaciones del ciclo de vida (estados PENDIENTE, APROBADA, RECHAZADA y COMPLETADA) y la integración simulada con ms-inventory a través de RestTemplate. Se utilizó JaCoCo para medir la cobertura, alcanzando un porcentaje superior al 60% en la capa de servicio.

En el BFF, las pruebas se concentraron en los controladores y servicios que orquestan la comunicación con los microservicios (User, Inventory, Orders y

Restock), además del componente de seguridad JwtService y el manejador global de excepciones. Mediante `@ExtendWith(MockitoExtension.class)`, `@Mock` e `@InjectMocks` se simulaban las respuestas de los clientes Feign hacia cada microservicio, verificando que el BFF transforme y propague correctamente las respuestas (`DtoApiResponse`) hacia el frontend, incluyendo los escenarios de error controlado. En total se cubrieron los cuatro controladores BFF (Inventory, Orders, Restock y User), sus respectivos servicios, la validación de tokens JWT y el manejo global de excepciones, reforzando la confiabilidad de la capa que actúa como única puerta de entrada pública del sistema.

Pruebas unitarias en el frontend

A nivel de frontend, se implementaron pruebas unitarias con Vitest y React Testing Library, alcanzando 69 pruebas distribuidas en 13 archivos que cubren las cuatro capas de la API (Inventory, Orders, Restock y User), el contexto global de usuario (`UserViewModel`), las seis páginas principales (Login, Register, Inventory, Orders, Restock y Profile) y los componentes compartidos (Navbar y ProtectedRoute). Las pruebas de API mockean axios para verificar que cada endpoint reciba la URL, el método HTTP y el header de autenticación correctos, mientras que las pruebas de componentes usan React Testing Library junto con user-event para simular interacciones reales (llenar formularios, hacer clic en botones, seleccionar opciones) y mockean los módulos de API y el contexto de usuario para aislar la lógica de presentación. Se prestó especial atención a los flujos críticos de negocio, como el login con persistencia de sesión en localStorage, la protección de rutas mediante ProtectedRoute, y las validaciones de formularios en Inventory y Orders antes de enviar datos al backend.

Monitoreo y observabilidad con Sentry

Para complementar la estrategia de calidad, se integró Sentry (autoalojado sobre GlitchTip) como herramienta de monitoreo y captura de errores en tiempo real en los microservicios y en el BFF, mediante las dependencias sentry-spring-boot y sentry-logback. Cada servicio se configuró con su propio DSN, entorno (production) y etiquetas (tags) que identifican la aplicación y el servicio de origen, lo que permite distinguir en el panel de Sentry desde qué microservicio proviene cada evento.

A nivel de código, la integración se realizó dentro de los GlobalExceptionHandler de cada microservicio, invocando `Sentry.captureException(ex)` en los manejadores de excepciones no controladas, de modo que cualquier error inesperado quede registrado automáticamente sin intervención manual. Se configuraron niveles mínimos de logging y de breadcrumbs (info, error y debug respectivamente) y una tasa de muestreo de trazas (traces-sample-rate) baja para no afectar el rendimiento en producción. Esto permitió detectar fallas durante la integración de servicios y las

pruebas del examen sin depender exclusivamente de la revisión manual de logs, aportando trazabilidad y aumentando la sostenibilidad operativa de la solución.

Estrategia de branching y gestión de versiones

El equipo utilizó una estrategia basada en ramas por funcionalidad (feature branches) sobre una rama principal (main/develop), donde cada integrante desarrollaba su microservicio o tarea en una rama independiente y luego integraba mediante pull requests. Esto permitió trabajar en paralelo en ms-restock, BFF y frontend sin bloquear a los demás integrantes, aunque en algunos casos generó conflictos de merge al modificar simultáneamente archivos de configuración compartidos (application.properties, pom.xml). Una mejora para futuras iteraciones sería adoptar convenciones más estrictas de nomenclatura de ramas y revisiones de código obligatorias antes de cada integración.

Documentación de APIs con Swagger / OpenAPI.

Con el fin de facilitar el consumo, prueba y mantenimiento de las APIs REST de cada microservicio, se implementó documentación automática mediante Swagger UI, basada en la especificación OpenAPI 3. Esto permite que cualquier desarrollador o evaluador pueda visualizar, probar y entender los endpoints disponibles sin necesidad de leer el código fuente ni levantar herramientas externas como Postman.

Se utilizó la librería springdoc-openapi (versión 3.0.2), compatible con Spring Boot 4, integrada mediante la dependencia springdoc-openapi-starter-webmvc-ui en el pom.xml de cada microservicio. Esta librería genera automáticamente la especificación Open API a partir de las anotaciones presentes en los controladores, y expone una interfaz web interactiva (Swagger UI) en la ruta /swagger-ui.html de cada servicio.

La documentación se implementó de forma independiente en los 5 microservicios del sistema: bff, ms-inventory, ms-orders, ms-restock y ms-user. Se creó una clase de configuración por microservicio, anotada con @OpenAPIDefinition, donde se define. Título, versión y descripción funcional de la API, Datos de contacto y licencia. Servidores disponibles (desarrollo local y despliegue interno vía Docker/Kubernetes). En cada controlador se agregaron las anotaciones, tag a nivel de clase, para agrupar los endpoints por dominio funcional, operation en cada método, describiendo su propósito. ApiResponse, detallando los posibles códigos de respuesta HTTP (200, 400, 404, 409, etc.).

El microservicio bff ya contaba con documentación Swagger previa a este trabajo, incluyendo un esquema de seguridad Bearer Authentication para reflejar el uso de tokens JWT en los endpoints protegidos.

JavaDoc.

Con el objetivo de mejorar la mantenibilidad, comprensión y documentación del proyecto, se incorporó JavaDoc en todos los microservicios desarrollados, exceptuando el componente Backend For Frontend (BFF). La documentación fue aplicada a las clases principales de cada microservicio, incluyendo controladores, servicios, entidades, DTOs, repositorios y clases de manejo de excepciones. Además, se documentaron los métodos públicos indicando su propósito, los parámetros recibidos y el valor retornado, permitiendo que otros desarrolladores comprendan fácilmente el funcionamiento del sistema sin necesidad de analizar la implementación completa.

La incorporación de JavaDoc aporta beneficios como facilita la comprensión del código fuente, mejora la mantenibilidad del sistema, favorece el trabajo colaborativo entre desarrolladores, permite generar documentación técnica automáticamente mediante las herramientas de Java, sigue las buenas prácticas recomendadas para proyectos desarrollados con Spring Boot.

El componente BFF no fue documentado mediante JavaDoc debido a que, tras su migración a KrakenD como API Gateway, ya no corresponde a un proyecto Java sino a una configuración declarativa (krakend.json), por lo que no existe código fuente Java sobre el cual aplicar JavaDoc en ese componente.